

## LeetCode Patterns Cheat Sheet (concise)

=====

### Key Patterns

-----

1. Sliding Window
  - Use for subarray/subsequence problems where you need to find optimal window by expanding/contracting.
2. Two Pointers
  - Useful for sorted arrays, removing duplicates, partitioning.
3. Fast & Slow Pointers
  - Detect cycles, find midpoints of linked lists.
4. Backtracking
  - Use for permutation, combination, subset and constraint-satisfaction problems.
5. Depth/Breadth First Search (DFS/BFS)
  - Trees & graphs; use recursion or stack for DFS, queue for BFS.
6. Divide & Conquer
  - Merge sort, binary search variants.
7. Dynamic Programming
  - Recognize overlapping subproblems; use memoization or tabulation.
8. Greedy
  - Choose locally optimal choices, then prove correctness.
9. Topological Sort
  - For scheduling/ordering with dependencies.
10. Binary Search on Answer
  - Use when searching a numeric answer over a monotonic function.

### Practice Strategy

-----

- Tag problems by pattern while solving.
- When stuck, map problem to a pattern instead of random exploration.
- Implement template solutions for common patterns and adapt.

### Example Templates (pseudo)

-----

Sliding window (max sum of size k):

```
for r in range(n):
    window_add(a[r])
    if r - l + 1 > k:
        window_remove(a[l])
        l += 1
    update_answer()
```

Two pointers (sorted array pair sum):

```
l, r = 0, n-1
while l < r:
    s = a[l] + a[r]
    if s == target: return (l,r)
    if s < target: l += 1
    else: r -= 1
```

### Closing

-----

Patterns simplify problem solving by turning unfamiliar problems into known templates. Practice mapping problems to patterns.